

Lab Report 8: Robot Modeling and Visualization via URDF

Submitted By: Syeda Imaan Fatima 2022-MC-28

Course: Mobile Robotics (MCT-454L)

Workspace: `~/ros2_ws_imaan`

1. Objective

The objective of this lab was to design and implement a custom differential-drive mobile robot using the **Unified Robot Description Format (URDF)**. Key goals included:

- Defining a 13-link kinematic tree using primitive geometries.
- Organizing the model into a standard ROS 2 `ament_cmake` package.
- Visualizing the robot and its **TF (Transform) frames** in RViz.
- Preparing the model with inertial and collision properties for future Gazebo physics integration.

2. Technical Background

2.1 The URDF Framework

URDF is an XML-based specification used to describe the physical and logical structure of a robot.

- **Links:** Represent rigid bodies with visual, collision, and inertial properties.
- **Joints:** Define the parent-child relationship and motion constraints (e.g., `fixed`, `continuous`).
- **Kinematic Tree:** A hierarchical structure where every link (except the `base_link`) has exactly one parent.

2.2 Coordinate Frames (TF2)

Each link in the URDF serves as a coordinate frame. The `robot_state_publisher` calculates the relative transformations ($\$X, Y, Z$ and $\$R, P, Y$) between these frames and broadcasts them as `/tf` messages. This allows RViz to render the robot accurately in 3D space.

3. Methodology & Implementation

3.1 Robot Architecture

I designed a 13-link robot featuring a chassis, sensor deck, and a multi-link antenna.

Feature	Link Name(s)	Joint Type	Geometry
Chassis	<code>base_link</code>	Root	Box (0.4m x 0.3m x 0.1m)
Drive	<code>left_wheel</code> , <code>right_wheel</code>	Continuous	Cylinder (Rotated 90°)
Stability	<code>caster_wheel</code>	Fixed	Sphere
Sensors	<code>camera</code> , <code>lidar</code>	Fixed	Cylinder
Decorative	<code>antenna_tip</code>	Fixed	Sphere

3.2 Procedure & Commands

1. **Package Setup:** Created the `my_robot_description` package using the `ament_cmake` build type to ensure proper installation of non-Python assets (URDF, Launch, RViz).
2. **Environment Preparation:**

```
sudo apt install ros-humble-joint-state-publisher-gui ros-humble-tf2-tools
```

3. **Build Process:** Used the `--symlink-install` flag to allow real-time edits to the URDF without needing to re-run `colcon build`.

```
colcon build --packages-select my_robot_description --symlink-install
```

```
source install/setup.bash
```

4. **Launch Visualization:** Executed the custom `display.launch.py` which orchestrated the `robot_state_publisher` and the `rviz2` node.

```
ros2 launch my_robot_description display.launch.py
```

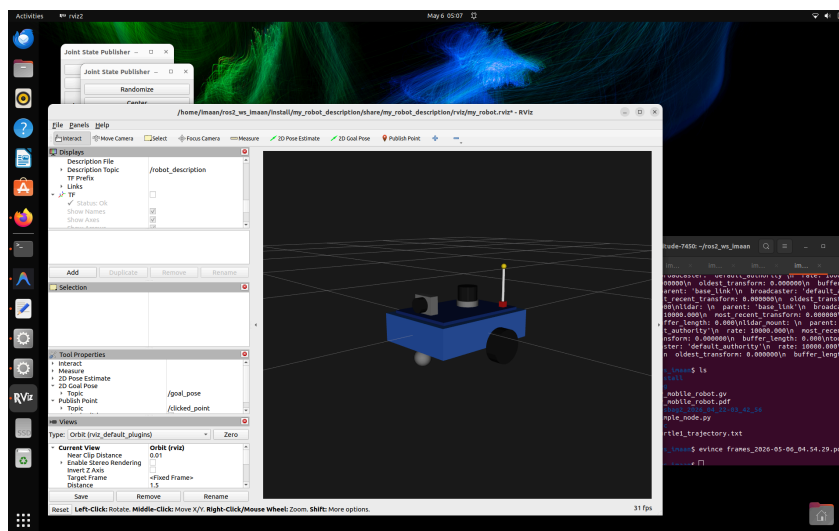

4.2 Joint Dynamics

Moving the sliders in the `joint_state_publisher_gui` resulted in immediate rotation of the wheel models in RViz. This confirmed the functional pipeline:

GUI Slider $\rightarrow$ `/joint_states` $\rightarrow$ `robot_state_publisher` $\rightarrow$ `/tf` $\rightarrow$ RViz Rendering.

4.3 Geometry Orientations

A critical finding during implementation was the **Cylinder Alignment**. By default, URDF cylinders align with their local Z-axis. To create wheels that rotate forward, I had to apply a 90° pitch/roll rotation (`rpv="1.5708 0 0"`) within the visual tag.



5. Design Customizations

To enhance the realism of the model, the following custom elements were added:

- **Antenna Chain:** A 3-link chain (`antenna_base` $\rightarrow$ `antenna` $\rightarrow$ `antenna_tip`) demonstrating chained fixed transformations.
- **Sensor Offsets:** The LiDAR was elevated on a `lidar_mount` pedestal to simulate a clear 360° Field of View (FOV).
- **Collision Bodies:** Every link was equipped with a `<collision>` tag identical to its visual geometry to ensure the robot can interact with walls/obstacles in simulation.

6. Conclusion

The lab successfully demonstrated the workflow for defining robot kinematics in ROS 2. The 13-link model is fully functional in RViz and, crucially, is **Gazebo-ready** due to the inclusion of `<inertial>` mass properties. The ability to manipulate the TF tree via joint states is a foundational step toward autonomous navigation and motion planning in the subsequent labs.

Deliverables Checklist

- ☐ **my_robot.urdf**: Full model description.
- ☐ **display.launch.py**: Node orchestration script.
- ☐ **frames.pdf**: Validated TF Tree structure.
- ☐ **Screenshots**: RViz visualization showing the RobotModel and TF axes.